

PITCH SCRIPT & PROJECT REFERENCE

Sentinel

Detect risk before it becomes loss.

Two risk-decisioning systems built around a single argument: the threshold that maximises a machine-learning metric and the threshold that maximises money saved are not the same number — and the gap between them is measurable in rupees.

Presenter: Ishan

Projects: Sentinel (return-risk decisioning) · Fraud Detector (PaySim transaction fraud)

Runtime: ~7 minutes of speaking + Q&A

Contents: Part 1 — Pitch script · Part 2 — Full project explanation · Part 3 — Q&A prep · Part 4 — Pre-pitch fixes

Part 1 The Pitch — Slide by Slide

Eight slides, roughly seven minutes. Amber blocks are the lines to say out loud; grey blocks are staging notes for you, not for the audience.

THE SPINE OF THE WHOLE PITCH

The strongest thing you have is not the model. It is this sentence, and everything else hangs off it: **“The threshold that maximises F1 and the threshold that maximises money saved are not the same number — and I can show you exactly how much that gap costs.”** Everyone else shows an ROC curve. Almost nobody shows a rupee curve.

SLIDE 1 The hook — one number, no context yet

~40s

SAY

“Optimising for F1 on my fraud model leaves one hundred and ten and a half million rupees of fraud on the table. Optimising for rupees leaves one point one million. Same model, same data — one number changed.”

“I'm Ishan. I built two risk systems to make that point.”

STAGING

Do not explain the number yet. Let it sit. The whole point is that it sounds impossible until slide 4.

SLIDE 2 The problem — business framing, not ML framing

~50s

SAY

“There are two sides to the same coin here. On the payments side, fraudulent transactions become chargebacks and direct loss. On the commerce side, returns cost a merchant three to four times the original shipping cost in reverse logistics, restocking and refund processing.”

“But structurally they are the *same decision*: given a signal, do I let this through, hold it, or escalate it — and what does being wrong cost me in each direction? That asymmetry is the whole game, and it's the part most ML projects never look at.”

SCREENSHOT

The dashboard. Put it up and name the one number on it that matters before you say anything else.

SAY

“Sentinel scores every order zero to one hundred *before fulfilment*. That score becomes a risk band — low, medium, high, critical — and high-risk orders route into a review queue where an operator approves, holds, or escalates, with every action written to an audit log.”

“End to end: synthetic dataset, three trained models, a FastAPI service, a Postgres-ready database, and a Next.js dashboard. Every number you see on that screen is computed from the actual trained model. Nothing on this dashboard is hardcoded.”

SCREENSHOT / CHART

The threshold simulator. If the chart doesn't already exist, build it: one x-axis of threshold, two curves — F1, and net rupees saved. The entire point is visual: the two peaks are in *different places*. This is the most important image in the deck.

SAY

“Here are two curves over the same threshold. The blue one is F1. The other is net rupees saved. They peak in different places.”

“On Sentinel, the F1-optimal threshold is fifty percent. The rupee-optimal threshold is fifteen. Moving to the rupee-optimal point saves about one lakh ninety-one thousand rupees against a baseline loss of five lakh ten — roughly thirty-seven percent.”

“On the fraud detector the gap is violent, because the costs are violently asymmetric. The average fraudulent transaction in my test set is about one and a half million rupees. A manual review costs fifty. A missed fraud costs about thirty-one thousand times more than a false positive. So the correct decision is to deliberately tank precision to nineteen percent in order to push recall to ninety-nine point three — because eighteen thousand unnecessary fifty-rupee reviews are cheap next to one more missed multi-lakh fraud.”

“F1 has no concept of money. That's why it gets this wrong.”

FRAUD DETECTOR — THRESHOLD	PRECISION	RECALL	FRAUD ₹ MISSED	TOTAL COST
0.500 — naive default	0.392	0.976	₹6,988,624	₹7,310,824
0.989 — F1-optimal	0.979	0.838	₹110,511,888	₹110,515,738
0.090 — cost-optimal	0.189	0.993	₹1,123,420	₹2,029,770

DO NOT SAY

Do not quote the “99.97% savings versus flagging nothing” figure. Your own README warns it is inflated by a handful of enormous transactions in the baseline sum. Anchor on the **₹2.03M residual cost** instead — it's the honest number, and saying it first disarms anyone who would have found the caveat themselves.

SCREENSHOT

Order detail with per-feature contributions, plus the review queue with the action bar.

SAY

“A bare probability with no reasoning is the single most common complaint against ML risk systems in practice — ‘why was my order flagged?’ So every prediction returns its top contributing factors: this transaction type pushed the score down by this much, this amount pushed it up by that much.”

“And the operator's decision isn't a mutable status field. An order's status is *derived* from its most recent audit-log entry. That means there is one source of truth, and the full history of who decided what is permanently reviewable. In a risk system that's not a nice-to-have — it's the thing a regulator or a post-mortem asks for.”

SAY

“This is the part I'm proudest of, and it's a story about catching myself.”

“PaySim's fraud generator always fully drains the origin account — ninety-nine point two percent of fraud rows end with the origin balance at exactly zero. A model given that feature scores a PR-AUC of nought point nine nine nine nine. Essentially perfect. It looks like the best result in the room.”

“Then I stress-tested it. I re-simulated all four thousand two hundred and fifty real fraud cases as if the fraudster had left just ten percent of the balance behind. The model's recall collapsed from ninety-seven point five percent to eight point eight. An eighty-nine point drop.”

“It was never detecting fraud. It was detecting ‘was this account drained to zero.’ So I removed the feature, retrained, and shipped the model that scores *worse* — PR-AUC nought point nine four five — because it's the one that actually survives a fraudster who changes behaviour. Its recall doesn't move at all across that test: ninety-nine point three percent, at every level.”

BALANCE LEFT BEHIND BY FRAUDSTER	LEAKY MODEL RECALL	SHIPPED MODEL RECALL
0% — the dataset's own behaviour	97.5%	99.3%
10%	8.8%	99.3%
25%	12.9%	99.3%
50%	16.4%	99.3%
75%	16.6%	99.3%

SAY — THEN RATTLE THESE OFF FAST, ONE LINE EACH

“Around that, six more checks most projects skip. **Calibration** — the raw model was badly overconfident, so isotonic regression cut the Brier score from nought point zero zero three eight to nought point zero zero zero six, an eighty-five percent improvement, which matters the moment anything downstream multiplies a probability by a rupee value. **Confidence intervals** — bootstrapped a thousand times, so recall is ninety-nine point three plus or minus half a point, not a point estimate pretending to be certain. **Per-segment thresholds** — TRANSFER fraud turns out to be nearly perfectly separable, so operationally, investigation effort belongs on CASH_OUT. **A temporal split, not a random one**, because the fraud rate on this data varies over time and shuffling would leak the future into training. **A decay test** across four subsequent weeks with no retraining. And **an unsupervised baseline** that I'm reporting as weak — PR-AUC nought point one two five — because it is weak, and talking it up would have been the easy dishonest move.”

SLIDE 7

Limitations — say them before you're asked

~40s

SAY

“Four honest limits. Both datasets are synthetic — this needs validation on real transaction data before anyone trusts the specific thresholds. The cost constants — forty rupees a review, twenty percent of order value per uncaught return, fifty-five percent of that recovered when caught — are labelled placeholders, not finance's real numbers. The models are static; there's no retraining pipeline or drift monitoring yet. And Sentinel's ROC-AUC is nought point seven, which is genuinely mediocre — I'll happily explain why that's deliberate.”

WHY THIS SLIDE WINS YOU POINTS

Naming your own weaknesses before the audience does converts every one of them from an ambush into evidence of judgement. It also sets up the best answer in your Q&A (see Part 3, first question).

SAY

“What I'd like: a conversation with the risk team. Specifically, I want to know which of my three cost assumptions I should replace with a real number first — because the moment those constants are real, every threshold in this system becomes a decision someone can actually act on.”

STAGING

End on a question you want them to answer, not on “thank you for your time.” It turns the close into the start of a conversation.

Where each screenshot goes

SCREENSHOT	SLIDE	THE ONE NUMBER TO SAY WHILE IT'S UP
Dashboard	3	“Every number here comes from the trained model.”
Threshold simulator	4	“F1 peaks at 50%. Rupees peak at 15%.”
Order detail — contributions	5	“This is the answer to ‘why was I flagged.’”
Review queue + action bar	5	“Status is derived from the audit log.”
Model performance page	Appendix	Only if asked.

Rule: never put a screenshot on screen without immediately saying the one number on it that matters. A screenshot with no narration is decoration.

Part 2 Complete Project Explanation

Everything behind the slides. Read this to know the system cold; you will only speak about a fraction of it, but the fraction you speak about will sound different because you know the rest.

Sentinel — return-risk decisioning

Purpose

Predict, *before an e-commerce order is fulfilled*, whether it will be returned — so an ops team can hold or verify the risky ones instead of absorbing reverse-logistics cost across the board. The prediction is not the product; the *routing decision* is the product.

Data

Fifteen thousand synthetic orders across eight product categories, with a roughly thirty-five percent return rate. Twenty-two raw features plus six derived ones, which become forty columns after encoding. Five percent deliberate label noise is injected so the model cannot reach an unrealistic ceiling — a synthetic dataset that a model can predict perfectly teaches you nothing.

ML pipeline

Stratified seventy / fifteen / fifteen split. Train three classifiers — Logistic Regression, Random Forest, Gradient Boosting. Select the best by F1 **on the validation set**. Evaluate exactly once on the held-out test set, which is never touched during training or model selection. Artefacts are pickled to `models/` alongside a metadata JSON, which is what the dashboard reads — so the UI cannot drift away from the model.

MODEL	PRECISION	RECALL	F1	ROC-AUC
Logistic Regression — selected	0.477	0.698	0.567	0.704
Random Forest	0.472	0.636	0.542	0.676
Gradient Boosting	0.444	0.735	0.553	0.666

Held-out **test-set** figures for the selected model, verified against the live `/api/metrics` endpoint. Train 10,500 · validation 2,250 · test 2,250 · 40 features. (`training_metadata.json` stores the *validation* numbers — 0.473 / 0.668 / 0.554 — which is what selection was made on. Different split, not a contradiction.)

The cost model — the piece that makes it a product

Three named assumptions live in one file, `ml/cost_config.py` : forty rupees per manual review; twenty percent of order value lost per uncaught return; fifty-five percent of that avoided when a return is caught and acted on, because intervention rarely stops a hundred percent.

The detail worth saying out loud: that same file is imported by *both* the offline threshold sweep and the live “rupees at stake” figure on the review queue. The two numbers are structurally incapable of disagreeing. That is a deliberate design decision against a very common class of bug where the analysis and the dashboard quietly diverge.

Result on the held-out test set: baseline loss if you flag nothing, ₹510,602. Best threshold by F1, fifty percent. Best threshold by rupees saved, fifteen percent. Net savings at the rupee-optimal threshold, ₹190,996 — about thirty-seven percent.

Backend

FastAPI with SQLAlchemy and Pydantic, on SQLite for development and Postgres-ready for production. Routes cover orders, predictions, metrics, alerts and the assistant. Two decisions worth defending in the room:

- **Status is derived, not stored.** `backend/review.py` computes an order's review status from its most recent audit-log row rather than from a mutable status column. One source of truth, complete history, and every operator action is permanently attributable.
- **The alerts module refuses to fabricate.** The synthetic data has no real time dimension — every row was seeded in one batch, and “order hour” is hour-of-day, not a timestamp. So the alerts deliberately do *not* claim “a spike in the last twenty-four hours,” which would be an invented number. They surface live risk-*concentration* signals instead, gated at a minimum sample of thirty and flagged medium at $1.3\times$ the platform rate, high at $1.75\times$.

Frontend

Next.js 14 with TypeScript, Tailwind and Recharts. Seven surfaces: dashboard, orders table (searchable, filterable, paginated), order detail with full risk breakdown, review queue with an action bar and action timeline, threshold simulator, model performance page, and the AI assistant.

AI assistant

Claude-backed with a rule-based fallback, grounded strictly in live application data. The grounding is the point: it answers questions about *this* model and *these* orders, so it cannot hallucinate a metric that the system doesn't actually have.

Testing

Twenty-one tests across the API, the data generator and the prediction path.

Fraud Detector — transaction fraud verification

Scope, stated defensively and on purpose

Trained on PaySim: 6.36 million mobile-money transactions, 8,213 fraud cases — a fraud rate of 0.13%, or roughly one in twelve hundred. The system is defence-only by construction: it scores a

transaction and returns a probability, a verdict at three thresholds, and a SHAP explanation. It has no network access and cannot move money or touch a live system. Worth stating in one sentence if the audience is a risk team; they will notice that you thought about it.

The split, and why it isn't random

Temporal: train on the earliest seventy percent of time steps, validate on the next ten, test on the final twenty — 1,248,736 transactions containing 4,250 frauds, never seen during training or threshold selection. A random shuffle would leak future fraud patterns backwards into training, and the fraud rate genuinely moves over time on this data — 0.08% in the train window versus 0.34% in the test window. A shuffled split would have handed you a better number and a worse model.

The leak, and the model you shipped instead

PaySim's generator drains the origin account to zero in 99.2% of fraud rows. A model handed that signal reaches PR-AUC 0.9999 with precision 1.000 and recall 0.999 — and it is worthless, because it learned “was this account emptied,” a proxy for the label rather than a property of fraud. The shipped model excludes that feature and scores PR-AUC 0.945 with precision 0.979 and recall 0.838 at its F1-optimal point. The adversarial test on slide 6 is the proof that the weaker number is the better model.

The eleven analysis scripts

Each answers exactly one question that most projects on this dataset skip:

SCRIPT	QUESTION IT ANSWERS
<code>explore.py</code>	Is the data what I think it is — fraud rate, type breakdown, split balance?
<code>check_leakage.py</code>	Is there a feature acting as a proxy for the label?
<code>train.py</code>	The full-feature model, kept only for reference.
<code>train_ablation.py</code>	The honest model, with the leaking feature removed. This is what ships.
<code>adversarial_test.py</code>	Does it survive a fraudster who changes behaviour?
<code>explain.py</code>	Why was this specific transaction flagged?
<code>calibration.py</code>	Does “70% probability” actually mean a 70% fraud rate?
<code>segment_thresholds.py</code>	Should TRANSFER and CASH_OUT have the same threshold?
<code>decay_over_time.py</code>	Does performance rot week over week without retraining?
<code>bootstrap_ci.py</code>	How certain are these numbers, really?
<code>isolation_forest_baseline.py</code>	What can be detected with no labels at all?

`predict.py` is the interactive CLI verifier and `api.py` exposes the same scoring logic over HTTP at `POST /score`, with interactive docs at `/docs`.

Two findings worth having ready

- **Calibration.** The class imbalance fix (`scale_pos_weight` on a 1:1225 ratio) helps ranking but distorts the probability's face value — transactions scored around 0.80 were actually fraud only 6.6% of the time. Isotonic recalibration on the validation set cut the Brier score from 0.00382 to 0.00058, an 84.7% improvement. It matters the instant anything downstream multiplies a probability by an amount to estimate expected loss.
- **Segmentation.** Per-type thresholds save only ₹7,500 — about 0.4% — over one global threshold. Small. But the finding underneath is operationally useful: **TRANSFER fraud is essentially solved, and nearly all remaining cost lives in CASH_OUT.** That tells an investigations team where to spend its week.

Part 3 Q&A — The Questions You Will Get

Answers written to be spoken, not read. Each begins by conceding the point, because conceding a fair point immediately is what makes the rest of the answer credible.

Q Your ROC-AUC is 0.70. That's weak.

“Correct — and it's a property of the data, not a bug I'm hiding. I injected five percent label noise into the generator specifically so the model couldn't look better than a real one would.”

“But the interesting result isn't the AUC. It's that even a nought-point-seven model saves thirty-seven percent of return cost once you tune the threshold on rupees instead of F1. A nought-point-nine-five model tuned on F1 would save less. That's the argument: threshold choice under asymmetric cost outperforms model quality here.”

Q Why did Logistic Regression beat XGBoost? That's suspicious.

“Because I generated the data with largely linear relationships, so a linear model matches the generative process almost exactly. It's an honest artefact of synthetic data and I'd expect the opposite on real messy merchant data.”

“I didn't hand-pick it, though — the pipeline selects by validation F1 automatically. Point it at real data and it'll pick the gradient-boosted model without me touching a line.”

Q It's synthetic data. What does any of this actually prove?

“It proves three things: the pipeline runs end to end, the cost framing is real, and — most importantly — it shows what I do when synthetic data lies to me.”

“On the fraud project I found the generator's full-drain artefact, proved it with an adversarial test, and then shipped the model that scored *worse* on every headline metric. That's the transferable skill. The model is disposable; the instinct to distrust a 0.9999 isn't.”

Q How would this go to production?

“Four things, in order. One: replace the three cost constants with finance's real numbers — that's a conversation, not an engineering task, and it changes every threshold in the system. Two: retrain on real orders. Three: add drift detection and a retraining pipeline; right now the model is static. Four: use the calibrated model, which I've already implemented on the fraud side via isotonic regression, because anything multiplying a probability by a rupee value needs the probability to be trustworthy, not just well-ranked.”

Q Why should a payments company care about e-commerce returns?

“Because it's the same decision architecture: score the event, threshold it on asymmetric cost, route to human review, log the action, keep it auditable. The fraud detector is that identical architecture pointed at transactions instead of orders. I built both to show the pattern transfers.”

Q Your cost-optimal threshold gives 19% precision. No ops team would accept flagging that much.

“That's the right challenge, and it's exactly why the fifty-rupee review cost needs to be a real number. If review capacity is the binding constraint rather than review cost, the optimisation changes shape — you'd solve for maximum recall subject to a fixed daily review budget instead of minimising total rupees.”

“The framework handles that; it's a different objective function over the same curve. What I'd need from you is the actual capacity number.”

Q How do you know the decay test means anything?

“It's deliberately narrow, and I'd flag that myself. It tests drift in volume and rate using the same *kind* of fraud each week — recall held around ninety-eight percent across four weeks with no retraining. It says nothing about fraud *tactics* evolving. That's a different question, and it's what the adversarial test exists for. Neither substitutes for the other.”

Part 4 Fix These Before You Pitch

Three concrete inconsistencies in the repo that would cost you credibility if a sharp listener caught them mid-pitch.

- ☐ **Your own interview-prep doc contradicts your model.** `ai-risk-manager/docs/interview-questions.md` says “a Gradient Boosting model” in both the thirty-second and two-minute pitches. But `training_metadata.json` and the README both record Logistic Regression as the selected model. If you rehearse from that doc you will state, out loud, a fact your own repository disproves. Fix the doc first.
- ☐ **Know the answer to the metrics question.** The README says precision 0.477 / recall 0.698 / F1 0.567; `training_metadata.json` says 0.473 / 0.668 / 0.554. These are *not* in conflict — the metadata holds **validation** metrics (what model selection ran on) and the README holds **held-out test** metrics (what the dashboard serves). If anyone spots the two numbers and challenges you, that distinction is the answer, and it's a point in your favour. **Always quote the test numbers.**
- ☐ **Retire the “99.97% savings” figure entirely.** It is technically correct and practically misleading — driven by a small number of very large transactions in the baseline sum, as your own README already admits. Anchor on the ₹2.03M residual cost. Anyone who reads the repo will find that caveat; you want to have said it before they do.

Final check, the night before

- ☐ The threshold simulator chart shows **rupees** on an axis, not just precision and recall. If it doesn't, this is the one thing to build.
- ☐ Both services actually start — backend on 8000, frontend on 3000 — from a cold boot, in under a minute.
- ☐ You can state, without looking: 50% versus 15%. ₹190,996. 37%. 0.989 versus 0.090. 31,000×. 97.5% → 8.8%.
- ☐ Both services boot cold: backend on 8000, frontend on 3000, and `/api/metrics` returns real numbers.
- ☐ You have one sentence ready for the ROC-AUC question, and you deliver it without apologising.

THE ONE THING TO REMEMBER IF YOU FORGET EVERYTHING ELSE

You are not pitching a model. You are pitching a way of thinking about risk decisions — that the metric which makes a model look good and the metric which makes a business money are different metrics, and that you know how to find the gap between them and measure it. Every number in this document is in service of that one sentence.

All figures drawn from the repository as it stands: `ai-risk-manager/` and `fraud-detector/`. Where the repo contradicts itself, Part 4 names which value to use.